

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский
Томский политехнический университет

Инженерная школа информационных технологий и робототехники
Отделение информационных технологий

Отчет по лабораторной работе №21 по дисциплине

«Язык Kotlin и основы разработки»

Сервисы

Выполнил:

Студент группы 1A22



О.К. Кравцов

Проверил:

Ст. преп. ОИТ ИШИТР



В.А. Дорофеев

Задание

Напишите программу-таймер. Интерфейс программы должен выглядеть примерно так:

Интервал:

Минуты:	Секунды:
0	30

Таймер готов к запуску

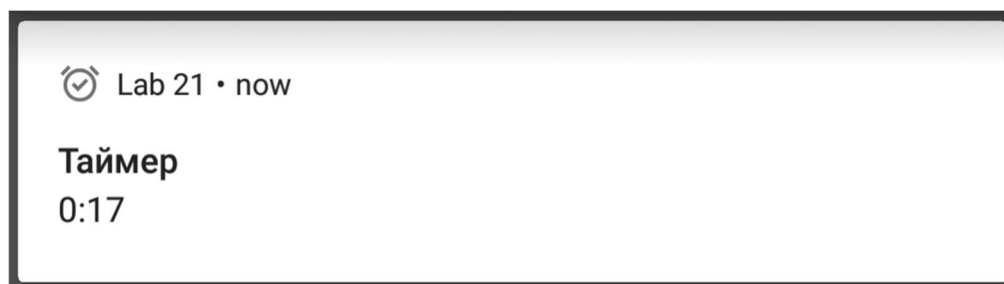
Пользователь может ввести числовые значения в поля минут и секунд. При нажатии кнопки «Пуск» начинается обратный отсчёт с тех значений, которые введены в поля минут и секунд, до нуля. На время отсчёта поля ввода блокируются, кнопка «Пуск» становится кнопкой «Стоп», а вместо надписи «Таймер готов к запуску» отображается оставшееся время:

Интервал:

Минуты:	Секунды:
0	30

0:25

Одновременно с этим отображается уведомление, в котором также идёт обратный отсчёт:



По окончании отсчёта программа возвращается в исходное состояние, готовая к новому запуску.

Если в процессе отсчёта пользователь закрывает программу, отсчёт должен продолжаться, уведомление должно оставаться активным (вместе с продолжающимся отсчётом), и при повторном запуске программы – отсчёт в окне программы также должен быть продолжен.

При реализации программы необходимо использовать сервис переднего плана, реализация отсчёта и уведомлений должны быть сделаны в сервисе.

Ход работы

1. Создан проект Lab21 на основе Empty Views Activity.
2. В файл манифеста добавлены разрешения для работы сервиса переднего плана и уведомлений:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <uses-permission android:name="android.permission.FOREGROUND_SERVICE" />
    <uses-permission android:name="android.permission.FOREGROUND_SERVICE_SPECIAL_USE" />

    <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.Lab21">

        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

        <service
            android:name=".TimerService"
            android:enabled="true"
            android:exported="false"
            android:foregroundServiceType="specialUse" />

    </application>
</manifest>
```

3. Настроены ресурсы приложения:

– strings.xml

```
<resources>
  <string name="app_name">Lab21</string>
  <string name="interval_label">Интервал:</string>
  <string name="minutes_label">Минуты:</string>
  <string name="seconds_label">Секунды:</string>
  <string name="start_button">Пуск</string>
  <string name="stop_button">Стоп</string>
  <string name="timer_ready">Таймер готов к запуску</string>
  <string name="set_timer_time">Установите время таймера</string>
  <string name="seconds_max_error">Секунды не могут быть больше 59</string>
  <string name="timer_finished">Таймер завершен!</string>
  <string name="notification_timer">Таймер</string>
  <string name="timer_started">Таймер запущен</string>
  <string name="timer_stopped">Таймер остановлен</string>
  <string name="timer_finished_notification">Таймер завершен</string>
  <string name="notification_remaining">Осталось: %s</string>
</resources>
```

– dims.xml

```
<resources>
  <dimen name="default_padding">16dp</dimen>
  <dimen name="vertical_margin">32dp</dimen>
  <dimen name="horizontal_margin_small">8dp</dimen>
  <dimen name="horizontal_margin_medium">32dp</dimen>
  <dimen name="edit_text_width">80dp</dimen>
  <dimen name="text_size_large">20sp</dimen>
  <dimen name="text_size_medium">18sp</dimen>
</resources>
```

4. Разработан пользовательский интерфейс:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/mainLayout"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="@dimen/default_padding"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/interval_label"
        android:textSize="@dimen/text_size_large"
        android:layout_gravity="center_horizontal"
        android:layout_marginBottom="@dimen/vertical_margin" />

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:gravity="center"
        android:layout_marginBottom="@dimen/vertical_margin">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/minutes_label" />

        <EditText
            android:id="@+id/minutesEditText"
            android:layout_width="@dimen/edit_text_width"
            android:layout_height="wrap_content"
            android:inputType="number"
            android:text="0"
            android:layout_marginStart="@dimen/horizontal_margin_small"
            android:layout_marginEnd="@dimen/horizontal_margin_medium" />

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/seconds_label" />

        <EditText
            android:id="@+id/secondsEditText"
            android:layout_width="@dimen/edit_text_width"
            android:layout_height="wrap_content"
            android:inputType="number"
            android:text="30"
            android:layout_marginStart="@dimen/horizontal_margin_small" />
    </LinearLayout>

    <Button
        android:id="@+id/startStopButton"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/start_button"
        android:layout_gravity="center_horizontal"
        android:layout_marginBottom="@dimen/vertical_margin" />

    <TextView
```

```
    android:id="@+id/timerStatusText"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/timer_ready"
    android:textSize="@dimen/text_size_medium"
    android:layout_gravity="center_horizontal" />
```

```
</LinearLayout>
```

5. Реализована основная активность MainActivity:

```
package ru.olegkravtsov.lab21

import android.content.ComponentName
import android.content.Context
import android.content.Intent
import android.content.ServiceConnection
import android.os.Build
import android.os.Bundle
import android.os.IBinder
import android.widget.Button
import android.widget.EditText
import android.widget.TextView
import android.widget.Toast
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
import androidx.core.app.ActivityCompat
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat

class MainActivity : AppCompatActivity() {
    private lateinit var minutesEditText: EditText
    private lateinit var secondsEditText: EditText
    private lateinit var startStopButton: Button
    private lateinit var timerStatusText: TextView

    private var timerService: TimerService? = null
    private var isBound = false

    private val connection = object : ServiceConnection {
        override fun onServiceConnected(className: ComponentName, service: IBinder) {
            val binder = service as TimerService.TimerBinder
            timerService = binder.getService()
            timerService?.setUpdateCallback { remainingSeconds ->
                runOnUiThread {
                    updateTimerDisplay(remainingSeconds)
                }
            }
            isBound = true
            updateUIFromService()
        }

        override fun onServiceDisconnected(className: ComponentName) {
            isBound = false
            timerService = null
        }
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_main)

        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.mainLayout)) { v,
insets ->
            val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
            v.setPadding(systemBars.left, systemBars.top, systemBars.right,
systemBars.bottom)
            insets
        }

        initViews()
        setupClickListeners()
    }
}
```



```

        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
            ActivityCompat.requestPermissions(
                this,
                arrayOf(android.Manifest.permission.POST_NOTIFICATIONS),
                1
            )
        }

        bindTimerService()
    }

    private fun initView() {
        minutesEditText = findViewById(R.id.minutesEditText)
        secondsEditText = findViewById(R.id.secondsEditText)
        startStopButton = findViewById(R.id.startStopButton)
        timerStatusText = findViewById(R.id.timerStatusText)
    }

    private fun setupClickListeners() {
        startStopButton.setOnClickListener {
            if (timerService?.isTimerRunning() == true) {
                stopTimer()
            } else {
                startTimer()
            }
        }
    }

    private fun bindTimerService() {
        val intent = Intent(this, TimerService::class.java)
        bindService(intent, connection, Context.BIND_AUTO_CREATE)
    }

    private fun startTimer() {
        val minutes = minutesEditText.text.toString().toIntOrNull() ?: 0
        val seconds = secondsEditText.text.toString().toIntOrNull() ?: 0

        if (minutes == 0 && seconds == 0) {
            Toast.makeText(this, getString(R.string.set_timer_time),
                Toast.LENGTH_SHORT).show()
            return
        }

        if (seconds >= 60) {
            Toast.makeText(this, getString(R.string.seconds_max_error),
                Toast.LENGTH_SHORT).show()
            return
        }

        val intent = Intent(this, TimerService::class.java)
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            startForegroundService(intent)
        } else {
            startService(intent)
        }

        timerService?.startTimer(minutes, seconds)
        updateUIForRunningState()
    }

    private fun stopTimer() {
        timerService?.stopTimer()
        updateUIForStoppedState()
    }

```

```

private fun updateUIFromService() {
    if (timerService?.isTimerRunning() == true) {
        val remainingSeconds = timerService?.getRemainingTime() ?: 0
        updateUIForRunningState()
        updateTimerDisplay(remainingSeconds)
    } else {
        updateUIForStoppedState()
    }
}

private fun updateUIForRunningState() {
    startStopButton.text = getString(R.string.stop_button)
    minutesEditText.isEnabled = false
    secondsEditText.isEnabled = false
}

private fun updateUIForStoppedState() {
    startStopButton.text = getString(R.string.start_button)
    minutesEditText.isEnabled = true
    secondsEditText.isEnabled = true
    timerStatusText.text = getString(R.string.timer_ready)
}

private fun updateTimerDisplay(remainingSeconds: Int) {
    if (remainingSeconds > 0) {
        val minutes = remainingSeconds / 60
        val seconds = remainingSeconds % 60
        timerStatusText.text = String.format("%d:%02d", minutes, seconds)
    } else {
        updateUIForStoppedState()
        Toast.makeText(this, getString(R.string.timer_finished),
Toast.LENGTH_SHORT).show()
    }
}

override fun onDestroy() {
    super.onDestroy()
    if (isBound) {
        unbindService(connection)
        isBound = false
    }
}
}

```

7. Реализация сервиса:

```
8. package ru.olegkravtsov.lab21

import android.app.Notification
import android.app.NotificationChannel
import android.app.NotificationManager
import android.app.Service
import android.content.Intent
import android.os.Binder
import android.os.Build
import android.os.IBinder
import androidx.core.app.NotificationCompat
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.Job
import kotlinx.coroutines.delay
import kotlinx.coroutines.launch

class TimerService : Service() {
    private val binder = TimerBinder()
    private var totalSeconds = 0
    private var remainingSeconds = 0
    private var isRunning = false
    private var timerJob: Job? = null
    private var updateCallback: ((Int) -> Unit)? = null

    private val notificationId = 1
    private val channelId = "timer_channel"

    inner class TimerBinder : Binder() {
        fun getService(): TimerService = this@TimerService
    }

    override fun onBind(intent: Intent): IBinder {
        return binder
    }

    override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int
    {
        return START_NOT_STICKY
    }

    fun setUpdateCallback(callback: (Int) -> Unit) {
        updateCallback = callback
    }

    fun startTimer(minutes: Int, seconds: Int) {
        if (isRunning) return

        totalSeconds = minutes * 60 + seconds
        remainingSeconds = totalSeconds
        isRunning = true

        startForegroundService()
        startTimerCountdown()
    }

    fun stopTimer() {
        isRunning = false
        timerJob?.cancel()
        updateNotification(getString(R.string.timer_stopped))
        stopSelf()
    }
}
```

```

fun getRemainingTime(): Int = remainingSeconds

fun isTimerRunning(): Boolean = isRunning

private fun startForegroundService() {
    createNotificationChannel()

    val notification = buildNotification(remainingSeconds,
getString(R.string.timer_started))

    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q) {
        startForeground(notificationId, notification,
        FOREGROUND_SERVICE_TYPE_SPECIAL_USE)
    } else {
        startForeground(notificationId, notification)
    }
}

private fun createNotificationChannel() {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        val channel = NotificationChannel(
            channelId,
            getString(R.string.notification_timer),
            NotificationManager.IMPORTANCE_LOW
        ).apply {
            description = "Канал для уведомлений таймера"
        }

        val notificationManager =
getSystemService(NotificationManager::class.java)
        notificationManager.createNotificationChannel(channel)
    }
}

private fun buildNotification(seconds: Int, status: String =
getString(R.string.notification_timer)): Notification {
    val timeText = formatTime(seconds)

    return NotificationCompat.Builder(this, channelId)
        .setContentTitle(status)

        .setContentText(String.format(getString(R.string.notification_remaining),
timeText))
        .setSmallIcon(android.R.drawable.ic_dialog_info)
        .setPriority(NotificationCompat.PRIORITY_LOW)
        .setOnlyAlertOnce(true)
        .setOngoing(true)
        .build()
}

private fun updateNotification(status: String =
getString(R.string.notification_timer)) {
    val notification = buildNotification(remainingSeconds, status)
    val notificationManager =
getSystemService(NotificationManager::class.java)
    notificationManager.notify(notificationId, notification)
}

private fun startTimerCountdown() {
    timerJob = CoroutineScope(Dispatchers.Main).launch {
        while (isRunning && remainingSeconds > 0) {
            delay(1000)
            remainingSeconds--
        }
    }
}

```

```

        updateNotification()
        updateCallback?.invoke(remainingSeconds)
    }

    if (remainingSeconds <= 0) {
        timerFinished()
    }
}

private fun timerFinished() {
    isRunning = false
    updateNotification(getString(R.string.timer_finished_notification))
    updateCallback?.invoke(0)

    CoroutineScope(Dispatchers.Main).launch {
        delay(3000)
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.N) {
            stopForeground(STOP_FOREGROUND_REMOVE)
        } else {
            @Suppress("DEPRECATION")
            stopForeground(true)
        }
        stopSelf()
    }
}

private fun formatTime(seconds: Int): String {
    val min = seconds / 60
    val sec = seconds % 60
    return String.format("%d:%02d", min, sec)
}

override fun onDestroy() {
    super.onDestroy()
    timerJob?.cancel()
    isRunning = false
}

companion object {
    private const val FOREGROUND_SERVICE_TYPE_SPECIAL_USE = 1 shl 30
}
}

```

Результат работы

При запуске отображается интерфейс с полями ввода минут и секунд, кнопкой «Пуск» и надписью «Таймер готов к запуску» (рис. 1).

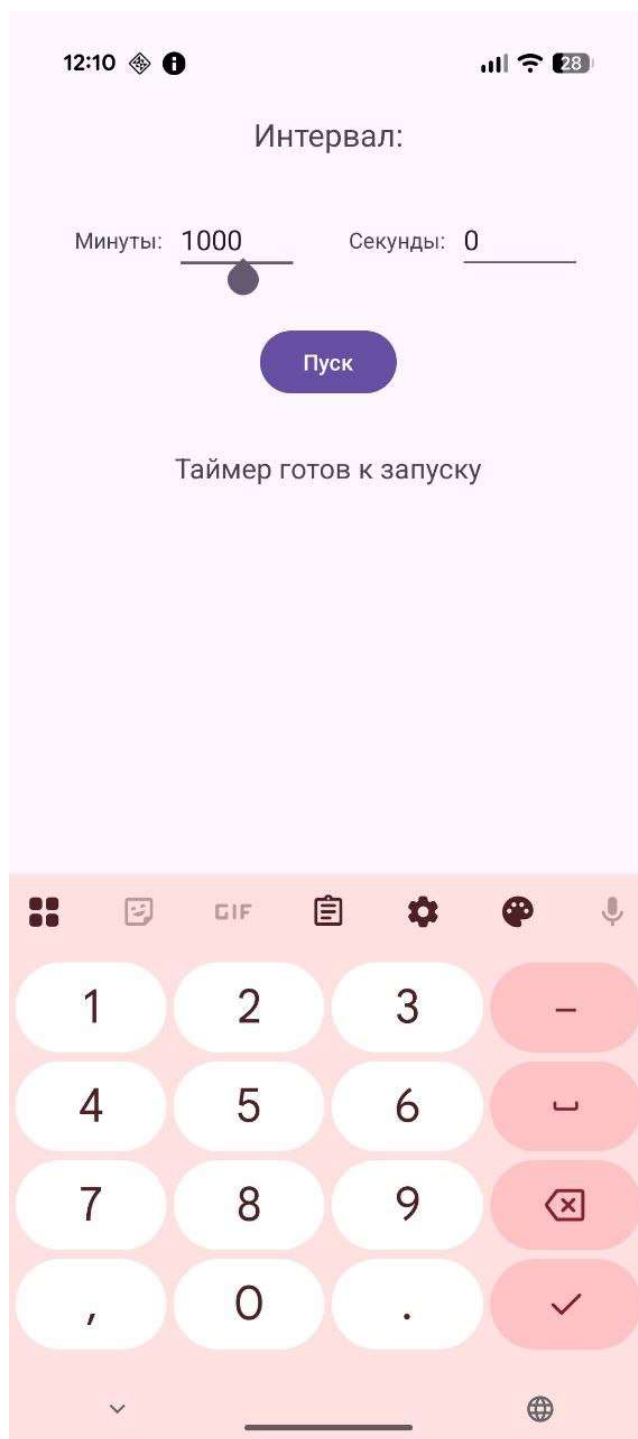


Рисунок 1 – Начальный экран приложения

После ввода времени и нажатия кнопки «Пуск» начинается отсчёт. Кнопка меняется на «Стоп», поля ввода блокируются, а надпись отображает оставшееся время (рис. 2).

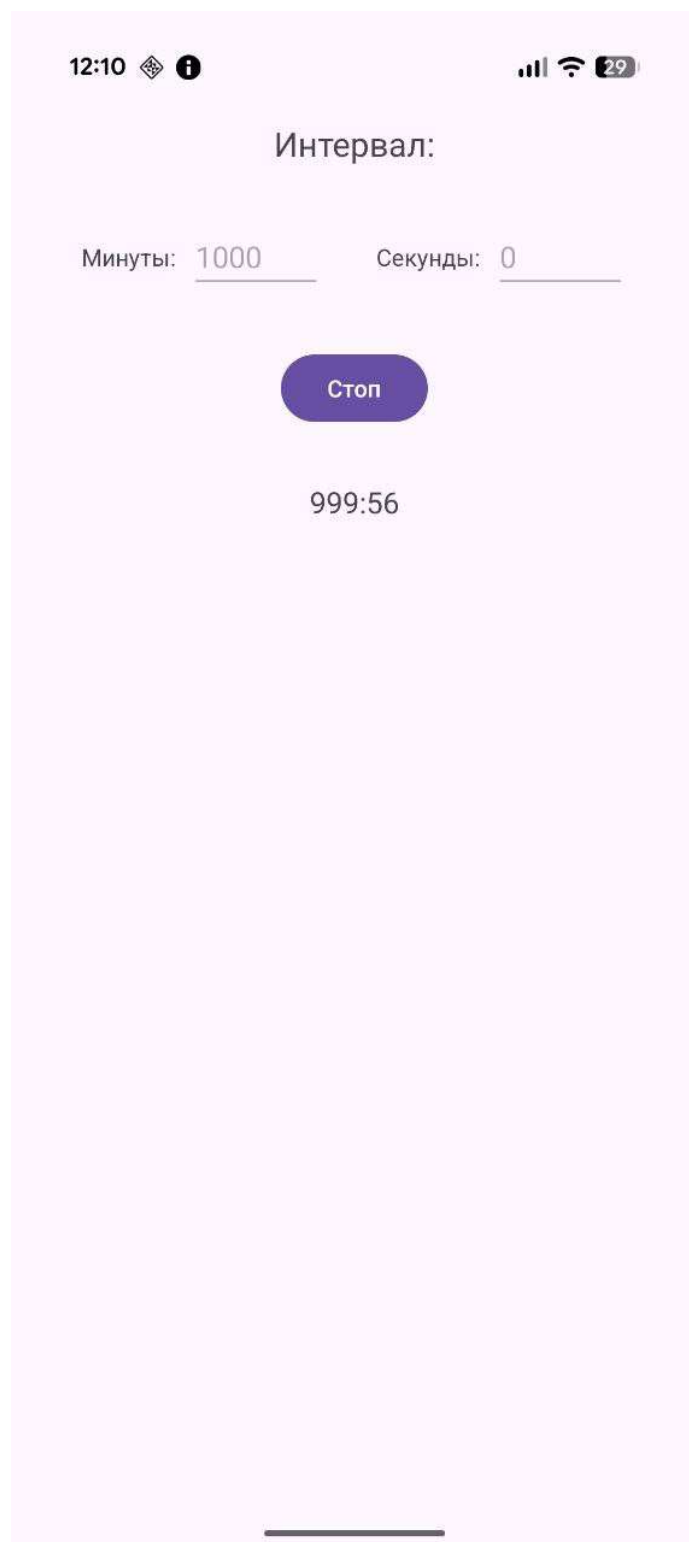


Рисунок 2 – Таймер запущен

Появляется уведомление с текущим оставшимся временем (рис. 3).

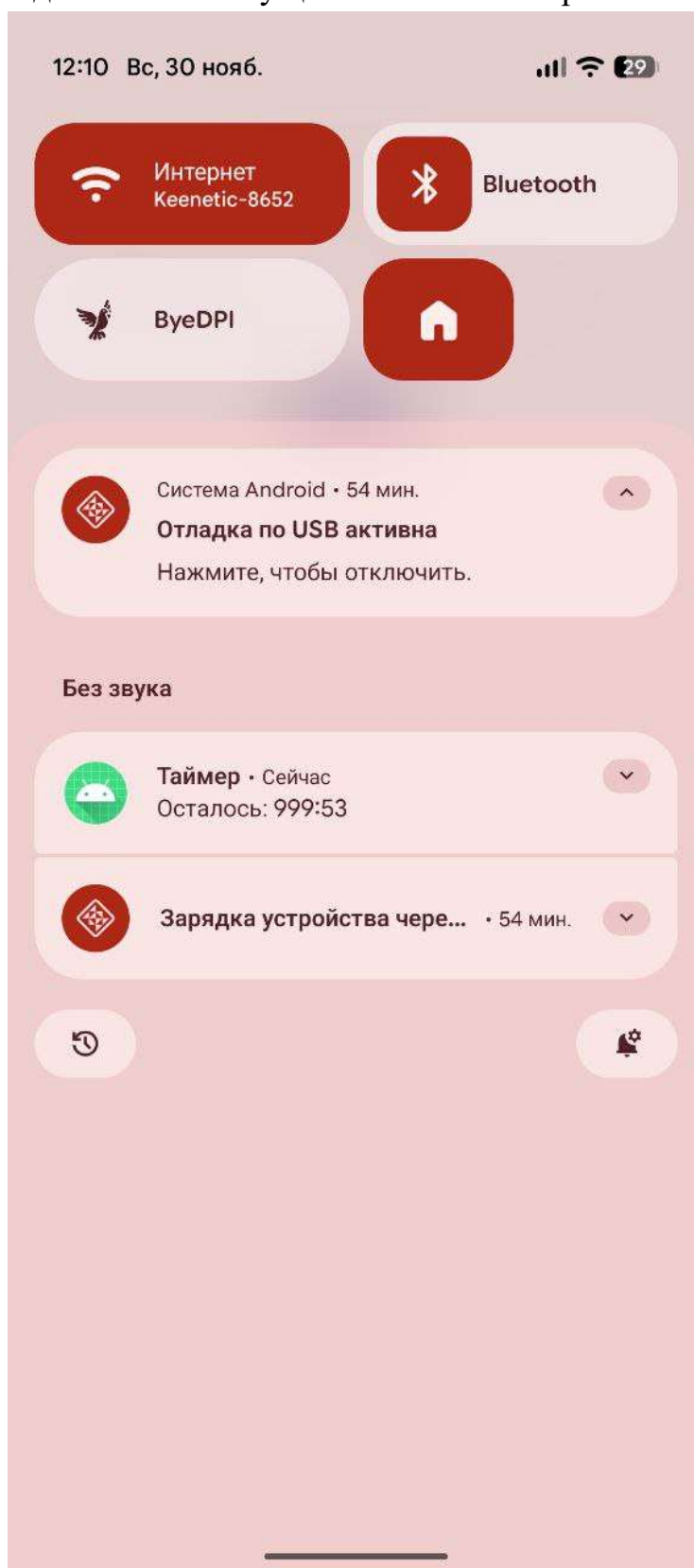


Рисунок 3 – Уведомление таймера

После завершения отсчёта уведомление меняет статус на «Таймер завершён», интерфейс возвращается в исходное состояние, а пользователь уведомляется сообщением (рис. 4).

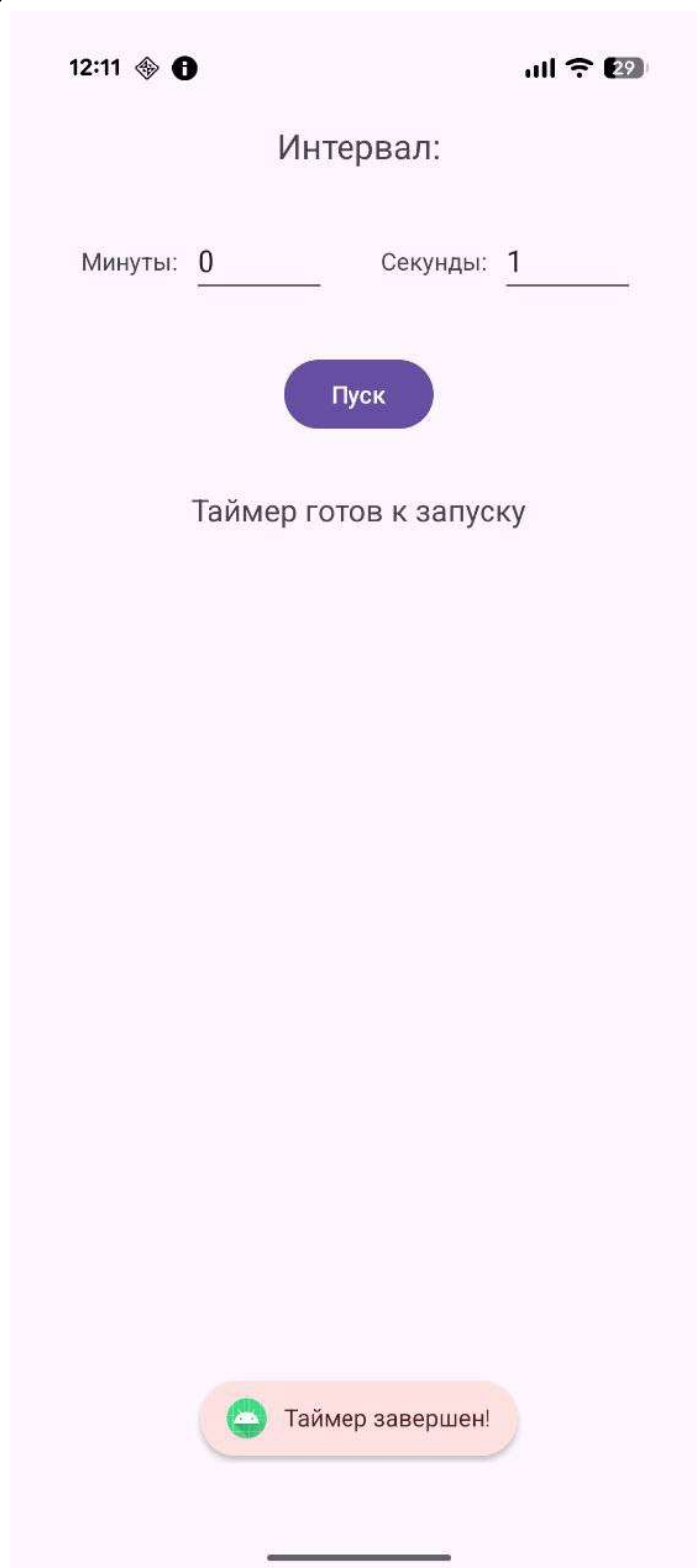


Рисунок 4 – Таймер завершён

При закрытии приложения таймер продолжает работу, а уведомление обновляется. Повторный запуск приложения синхронизирует интерфейс с текущим состоянием таймера.

Выводы

В ходе лабораторной работы было разработано приложение-таймер для Android, использующее сервис переднего плана. Основной задачей являлось обеспечение работы обратного отсчёта даже после закрытия приложения, что было успешно реализовано благодаря Foreground Service. В процессе работы освоены ключевые аспекты создания и управления сервисами, включая их привязку к активности, организацию длительных фоновых операций и взаимодействие через интерфейс IBinder. Также была настроена система уведомлений с постоянным обновлением информации о состоянии таймера. Полученный результат подтвердил эффективность использования сервисов для выполнения задач, требующих непрерывной работы.